

Post-CTS Delay Insertion to Fix Timing Violations

Baris Taskin and Jianchao Lu
Electrical and Computer Engineering Department
Drexel University
Philadelphia, PA 19104
Email: taskin@coe.drexel.edu, jl597@drexel.edu

Abstract—In mainstream ASIC design, industry standard automation tools are used in generating circuit implementations that satisfy the timing and power budgets. A typical timing budget follows the specifications of a clock frequency governed by the longest data path in the circuit. To satisfy this constraint, a zero-skew clock network that minimizes or bounds the clock skew is synthesized. Due to variations, however, zero clock skew cannot always be maintained and timing violations occur. This paper describes a post-clock-tree synthesis (CTS) delay insertion process on the clock tree network in order to fix timing violations that occur after such automated design tools are used. A mathematical formulation is presented which computes the minimum amount of delay to be inserted on each branch of the clock network. Experimental results show that the clock networks of the largest ISCAS'89 circuits can be corrected post-CTS to resolve the timing conflicts in approximately 90% of the circuits with minimal delay insertion ($0.159 \times \text{clock period}$ per clock path on average).

I. INTRODUCTION

The performance enhancing effects of clock skew scheduling has been known since 1990s [1]. However, due to the scalability of application methods and the ease of zero-skew design, non-zero clock skew design has not been integrated into the mainstream design process. Integrated circuits (ICs) are implemented using industry-standard automation tools that are optimized for zero-skew clock systems. The longest data propagation delay is selected as the operating frequency, which guarantees violation-free operation assuming a zero clock skew clock network. Consequently, the goal of the automated clock design tools is to either bound or minimize clock skew.

As the minimum feature size of VLSI circuits continues to shrink, however, process variations have become significantly worse [2]. The amount of variations in process parameters such as transistor channel length and threshold voltages can be represented as a growing percentage of their nominal values. The delay variations on clock network branches, for instance, correspond to 10% deviation from their nominal value for deep sub-micron technologies [3]. This trend for global skew mismatches in recent microprocessors has been well documented [4].

In this paper, a practical delay insertion process to be performed on a synthesized clock tree is introduced. This process is devised to work with industry standard automation tools, such that, the clock distribution network (*i.e.* clock tree) and the placement and routing results as generated by the tools are collected as inputs. A standard timing verification tool is used to detect the violations on the data paths. These violations

are eliminated (*i.e.* fixed) by inserting small delay elements on the clock branches. The process keeps the placement, routing and most of the clock tree synthesis (CTS) results of the automated results intact. Thus, valuable design optimizations embedded in the clock design automation process, such as crosstalk elimination [5] and leakage prevention [6], are mostly preserved. Note that, traditional clock skew scheduling, such as [1], would often require clock tree redesign. A linear programming (LP) formulation is presented to model and optimally solve the proposed post-CTS delay insertion process.

The rest of the paper is organized as follows. In Section II, the clock distribution network of an IC (particularly a clock tree) is presented as relevant to this paper. In Section III, the static timing constraints (hold and setup time) are reviewed. In Section IV, the potential timing violations due to clock delay variations are shown and the use of post-CTS delay insertion methodology to circumvent these constraints is described. In Section V, experimental results on a suite of ISCAS'89 benchmark circuits with randomly generated clock networks are presented. The paper is finalized in Section VI.

II. STRUCTURE OF A CLOCK TREE

Clock network design (also called clock tree synthesis) [7] is an essential step in the physical design flow of integrated circuits. During the clock network design step, the interconnect topology of the clock distribution network is designed based on the placement and routing information. The clock distribution network is frequently organized as a rooted tree structure [8,9], as illustrated in Fig. 1. A circuit schematic of a clock distribution network is shown in Fig. 1(a). An abstract graphical representation of the tree structure depicted in Fig. 1(a) is shown in Fig. 1(b). The unique source of the clock signal is at the root of the tree. This signal is distributed from the source to every register in the circuit through a sequence of *buffers and interconnect wires*. Such minimal or zero clock skew can be achieved by different routing strategies [10–13], buffered clock tree synthesis, symmetric n -ary trees [14] (most notably H-trees), using deskew buffers [15] or a distributed series of buffers connected as a mesh [16]. In this work, a generic tree implementation as shown in Fig. 1 is considered.

III. STATIC TIMING CONSTRAINTS

A local data path $R_i \rightarrow R_f$ consists of two registers $R_{i(initial)}$ and $R_{f(final)}$ and a combinational logic block. The minimum and maximum propagation delays on the combinational block

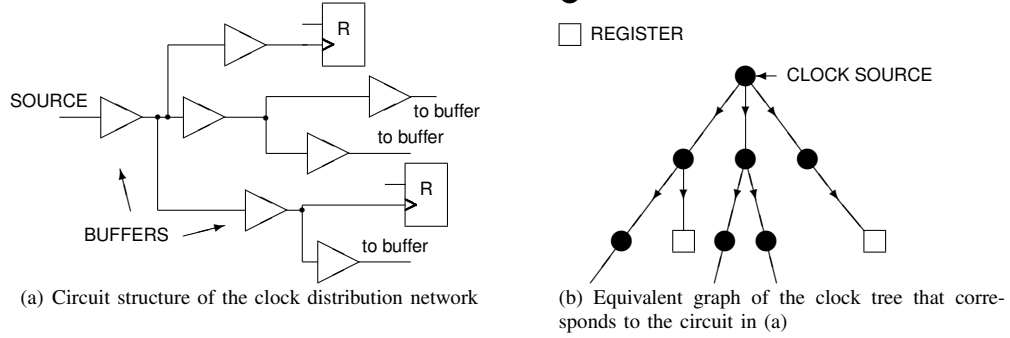


Fig. 1. Tree structure of a clock distribution network.

are denoted by D_{Pmin}^{if} and D_{PMax}^{if} , respectively. The clock-to-output delay of a register R_i is denoted by D_{CQ}^i , whereas the setup and hold times are denoted by S^i and H^i , respectively. Due to process parameter variations or by design, clock (signal) delays at each synchronous component of a circuit may be nonidentical. Clock skew is defined as $T_{skew}^{if} = t_i - t_f$, where t_i and t_f represent the clock delays to registers R_i and R_f , respectively. The clock period is denoted by T .

The timing analysis of a synchronous circuit is performed by satisfying the *setup* and *hold* timing constraints for each local data path:

$$\text{Setup: } t_i + D_{CQ}^i + D_{PMax}^{if} \leq t_f + T - S^f \quad (1)$$

$$\text{Hold: } t_i + D_{CQ}^i + D_{Pmin}^{if} \geq t_f + H^f. \quad (2)$$

For zero clock skew systems, clock delays t_i and t_f are identical, simplifying the timing constraints. Further assuming that the internal register delays can be neglected ($D_{CQ} = S = H = 0$), a limitation on the clock period T is derived from (1):

$$D_{PMax}^{if} \leq T. \quad (3)$$

Thus, if the circuit operates at any clock period less than the largest maximum data propagation time, a *timing violation* occurs [17]. Finding a clock period T for a zero clock skew circuit is always possible, making it convenient to design zero clock skew systems. Consequently, the application of zero clock skew schemes has been central to the design of fully synchronous digital circuits for decades [8].

IV. POST-CTS DELAY INSERTION

In mainstream IC design, automated place & route and CTS tools are used to compute the physical implementation of the circuit. The logic and memory elements are placed and routed with timing, congestion and power driven objectives. The clock tree is implemented in one of the forms described in Section II to deliver identical delays to each synchronous component. Despite aggressive optimizations, however, the clock network is still subject to random (and systematic) variations. These variations cause small shifts in the clock delays, leading to skew mismatches and potentially timing violations. We propose a delay insertion method to be performed after

the clock tree synthesis step (post-CTS) to fix the timing violations. Our problem definition is:

Given a pre-computed placement and routing, and a synthesized clock tree of an IC (thus, given the clock delay t_i of each branch, clock period T , local data path propagation time $[D_{Pmin}, D_{PMax}]$ of each local data path, internal register delays S , H and D_{CQ} of each register), compute the minimum amount of delay Δ_i to be inserted on each clock tree branch in order to eliminate timing violations, considering upper bounds for delay insertion per branch and total overall delay insertion.

Note that, typically the last stage buffers of clock trees (gaters) drive more than one register. The presented formulation here can easily be changed to reflect this requirement.

The mathematical formulation for the solution to this problem is derived as an LP form. After post-CTS delay insertion, (1) and (2) can be written as:

$$\text{Setup: } t_i + \Delta_i + D_{CQ}^i + D_{PMax}^{if} \leq t_f + \Delta_f + T - S^f \quad (4)$$

$$\text{Hold: } t_i + \Delta_i + D_{CQ}^i + D_{Pmin}^{if} \geq t_f + \Delta_f + H^f, \quad (5)$$

where the added term Δ_i is the delay element on clock tree branch driving R_i . We also assume two practical limitations on the delay insertion process. First, we assume that the amount of delay to be inserted on a clock tree branch has an upper bound proportional to the overall clock period T :

$$\Delta_i \leq k_1 T, \quad (6)$$

where k_1 is a design parameter. Second, we assume that the total amount of delay to be inserted ($\sum_{i=0}^N \Delta_i$) has an upper bound proportional to the clock period T and the number of registers N in the circuit:

$$\sum_{i=0}^{N-1} \Delta_i \leq k_2 TN, \quad (7)$$

where k_2 is a design parameter. In practical implementation, k_1 and k_2 can be determined by evaluating physical design information such as the area utilization, the number of clock tree levels and the power dissipation budget. Note that these constraints prevent excessive delay insertion, such as potentially

TABLE I
LP MODEL FOR POST-CTS DELAY INSERTION METHOD.

Minimize inserted delay	
min	$\sum_{i=0}^{N-1} \Delta_i$
s.t.	$t_i + \Delta_i + D_{CQ}^i + D_{PMax}^{if} \leq t_f + \Delta_f + T - S^f, \forall R_i \rightarrow R_f$ $t_i + \Delta_i + D_{CQ}^i + D_{Pmin}^{if} \geq t_f + \Delta_f + H^f, \forall R_i \rightarrow R_f$ $\Delta_i \leq k_1 T, \forall R_i$ $\sum_{i=0}^{N-1} \Delta_i \leq k_2 TN$

computed by a clock skew scheduling application. Although optimality is sacrificed, significantly reduced impacts on area and power are expected.

The LP model is shown in Table I. The objective is to minimize the total amount of delay insertion. The first two set of constraints are the setup and hold time constraints, respectively, defined for each local data path. The third set of constraints is the delay insertion upper bounds given in (6) defined for each clock branch. The fourth constraint is the total delay insertion bound given in (7).

The LP problem formulation guarantees minimum delay insertion. For instance, if no delay insertion is necessary, Δ_i evaluates to zero. For some circuits, the LP might return infeasibility which means either the timing violations cannot be resolved with the proposed delay insertion upper bounds or the circuit has *reconvergent paths* which are pathological cases [18] that cannot be solved with clock delay manipulation. Otherwise, the minimal amount of delay to be inserted on each branch is returned as a continuous variable. A more detailed integer linear programming problem (ILP) formulation can also be devised to model discrete values of delay to be inserted for a higher practical purpose.

The prescribed topology of a simple reconvergent path is shown in Fig. 2. For *some* such systems, timing violations

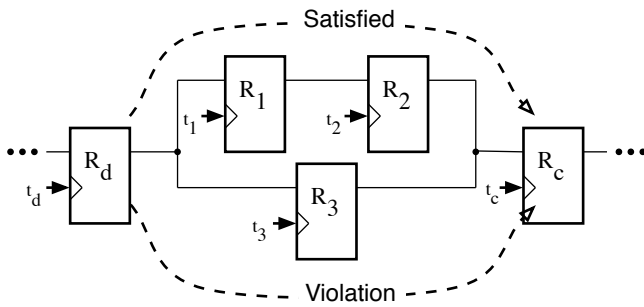


Fig. 2. A sample reconvergent path system. Clock delays t_d and t_c satisfy the timing of paths $R_d \rightarrow R_1, R_1 \rightarrow R_2, R_2 \rightarrow R_c$. However, the timing of one or both of paths $R_d \rightarrow R_3, R_3 \rightarrow R_c$ are violated.

cannot be resolved by manipulating clock delay values as the timing of both branches depends on the clock delays at the divergent R_d and convergent R_c registers. As presented in [18], in such cases, *delay insertion into the logic network or reduction of clock frequency* is necessary.

V. EXPERIMENTAL RESULTS

Proposed post-CTS delay insertion method is used in experiments on a suite of ISCAS'89 benchmark circuits for demonstration purposes. The timing information for ISCAS'89 circuits is generated with a pre-determined algorithm, in which the fanout, size and type of logic gates are considered. A single phase clock signal with a 50% duty cycle is selected for synchronization. The internal register delays (S, H, D_{CQ}) are assumed negligible. The clock network is also built experimentally; it is assumed that the clock period is selected as the largest data propagation delay in the circuit [which is typical in ASIC design, see (3)], the clock delay t_i to each register is arbitrarily selected to be $4T$ with a 10% variation. Upper bounds of post-CTS clock delay insertion is set to be $0.8T$ on each branch ($k_1 = 0.8$) with a total delay insertion of $0.4TN$ ($k_2 = 0.4$). In a real application, all experimental assumptions can be changed according to automated tool results.

The experiments are performed on a 2GHz Intel x86 processor with 1 GB RAM. The simplex optimizer of the GNU LP solver GLPK (version 4.21) [19] is used to solve the LP problems. The results are presented in Table II. In Table II, circuit information, zero clock skew operation frequency, timing violation data and post-CTS delay insertion data are presented. The number of registers and paths are shown in columns marked N and $\#paths$, respectively. The clock period T is selected as the largest data propagation delay in the circuit as derived in (3) to be functional for zero clock skew operation. The number of paths returning timing violation are shown in $\#p^{vio}$. The percentage of paths with timing violation is shown in $\%p^{vio}$. The total amount of violation (on all paths) is shown in column $\sum vio$. Post-CTS inserted delay information is presented in the last three columns, $\sum \Delta$ is the total inserted delay, $(\sum \Delta)/N$ is the average inserted delay per branch (register), and *metric* is a metric we established as average delay inserted per clock period, i.e. $(\sum \Delta)/(NT)$. We use the metric as an arbitrary measure of inserted delay density, as the delay values increase regardless of the circuit complexity as the clock period increases.

It is observed in Table II that, post-CTS delay insertion on the clock network is applicable to all circuits except for *s9234* on the selected suite of circuits. Due to the 10% variations in delays—which are randomly generated in experimentation—timing violations occur on 15.3% of the paths but as many as 80% of all the paths (*s1488*) and as low as 1 path (*s1196*) for a given circuit. The upper bounds of clock delay insertion, set by $k_1 = 0.8$ and $k_2 = 0.4$, enable us to fix the timing violations in most of the circuits by minimal delay insertion. The selected metric for delay insertion density

$$Metric : (\sum \Delta) / (NT) \quad (8)$$

has an average of 15.9%, which is reasonably small for practical implementation.

The timing violations for benchmark circuit *s9234* cannot be resolved with post-CTS delay insertion because of *reconvergent paths* [18]. Although not observed in our experiments,

TABLE II
POST-CTS DELAY INSERTION RESULTS FOR A SUITE OF ISCAS'89 BENCHMARK CIRCUITS.

Circuit Info			Clock	Violation Data			Insertion Data		
Circuit	N	$\#paths$	T	$\#p^{vio}$	$\%p^{vio}$	Σvio	$\Sigma \Delta$	$(\Sigma \Delta)/N$	Metric
<i>s1196</i>	18	20	20.8	1	5.0%	0.5	10.8	0.6	2.8%
<i>s1423</i>	74	1471	92.2	77	5.2%	1070.7	1531.8	20.7	22.5%
<i>s1488</i>	6	15	32.2	12	80.0%	80.8	61.2	10.2	31.7%
<i>s6669</i>	239	2138	128.6	68	3.2%	1630.2	6811.5	28.5	22.1%
<i>s9234</i>	228	247	75.8	86	34.8%	2204.3	N/A	N/A	N/A
<i>s13207</i>	669	3068	85.6	81	2.6%	1020.5	6279.2	9.4	11.0%
<i>s15850</i>	597	14257	116.0	406	2.8%	5025.4	11581.8	19.4	16.7%
<i>s15850.1</i>	534	10830	81.2	774	7.1%	8580.2	5767.2	10.8	13.3%
<i>s35932</i>	1728	4187	34.2	794	19.0%	6224.5	11232.0	6.5	19.0%
<i>s38417</i>	1636	28082	69.0	2206	7.9%	18811.2	12270.0	7.5	10.9%
<i>s38584</i>	1452	15545	94.2	178	1.1%	1837.1	34412.4	23.7	25.2%
<i>Average</i>					15.3%		8314.6	10.9	15.9%

the maximum delay insertion bounds on each clock branch $\Delta_i \leq k_1 T$ and the total delay insertion constraint $\Sigma \Delta < k_2 T N$ can also be limiting. For such circuits, designers can choose to follow typical procedures of performing iterative runs of placement, routing (or synthesis) to satisfy the specified timing budget. When such practices are costly, frequency specification can be relaxed to have the IC operate at a lower speed.

VI. CONCLUSIONS

The post-CTS clock delay insertion method has the same motivation as traditional clock skew scheduling in manipulating the clock delays in order to provide higher performance. However, it is different in the following aspects:

- Clock skew scheduling is performed before the clock network is synthesized. Our method is performed post-CTS, following a physical design implementation.
- Clock skew scheduling permits arbitrary clock delay values, which might be hard to synthesize. Our method starts off with the CTS results and only permits minimal delay insertion to keep the clock delays easy to realize.
- Clock skew scheduling cannot be used in conjunction with conventional automated design tools (e.g. automated tools do not support prescribed clock delays as a starting point in CTS). Our method can easily be interfaced with conventional automated design tools (e.g. automated results feed directly into the mathematical framework).
- The complexity of computation for clock skew scheduling is higher as clock period is a variable to be optimized (e.g. $\min T$). In our formulation, clock period is provided by the design specifications, thus, reducing complexity.

In experimentation with randomly generated ISCAS'89 clock networks, it is found that 10% variation in delay values of the clock network can result in 15.3% of the timing paths to fail the timing requirements. The post-CTS delay insertion method with practical limitations on inserted delay values (per path and per circuit) is used to solve the timing violations. This method successfully resolved the violations in 10 of the 11 experimented circuits. In practice, post-CTS delay insertion method can be used by designers to find quick solutions to timing violation problems without having to go through lengthy synthesis-placement-routing iterations.

REFERENCES

- [1] J. P. Fishburn, "Clock skew optimization," *IEEE Transactions on Computers*, vol. C-39, no. 7, pp. 945–951, July 1990.
- [2] S. R. Nassif, "Modeling and analysis of manufacturing variations," in *Proceedings of the IEEE Custom Integrated Circuits Conference*, 2001, pp. 223–228.
- [3] A. Kahng, "A roadmap and vision for physical design," in *Proceedings of the IEEE International Symposium on Physical Design (ISPD)*, 2002, pp. 112–117.
- [4] A. V. Mule, E. N. Glytsis, T. K. Gaylord, and J. D. Meindl, "Electrical and optical clock distribution networks for gigascale microprocessors," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 10, no. 5, pp. 582–594, October 2002.
- [5] J. Lou and W. Chen, "Crosstalk-aware placement," *IEEE Design & Test of Computers*, vol. 21, no. 1, pp. 24–32, Jan-Feb 2004.
- [6] A. Gupta, N. Dutt, F. J. Kurdahi, K. S. Khouri, and M. S. Abadir, "Leaf: A system level leakage-aware floorplanner for SoCs," in *Proceedings of the Asia and South Pacific Design Automation Conference*, 2007, pp. 274–279.
- [7] Q. K. Wu, *High-Speed Clock Network Design*. Kluwer Academic Publishers, 2003.
- [8] E. G. Friedman, *Clock Distribution Networks in VLSI Circuits and Systems*. IEEE Press, 1995.
- [9] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 2nd ed. MIT Press, 2001.
- [10] M. A. B. Jackson, A. Srinivasan, and E. S. Kuh, "Clock routing for high-performance ICs," *Proceedings of the ACM/IEEE Design Automation Conference*, pp. 573–579, June 1990.
- [11] R.-S. Tsay, "An exact zero-skew clock routing algorithm," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. CAD-12, no. 2, pp. 242–249, February 1993.
- [12] N.-C. Chou and C.-K. Cheng, "On general zero-skew clock net construction," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. VLSI-3, no. 1, pp. 141–146, March 1995.
- [13] N. Ito, H. Sugiyama, and T. Konno, "ChipPRISM: Clock routing and timing analysis for high-performance CMOS VLSI chips," *Fujitsu Scientific and Technical Journal*, vol. 31, no. 2, pp. 180–187, December 1995.
- [14] N. Gaddis and J. Lotz, "A 64-b quad-issue CMOS RISC microprocessor," *IEEE Journal of Solid-State Circuits*, vol. SC-31, no. 11, pp. 1697–1702, November 1996.
- [15] S. Rusu and G. Singer, "The first IA-64 microprocessor," *IEEE Journal of Solid-State Circuits*, vol. 35, no. 11, pp. 1539–1544, November 2000.
- [16] W. J. Bowhill *et al.*, "Circuit implementation of a 300-MHz 64-bit second-generation CMOS alpha CPU," *Digital Technical Journal*, vol. 7, no. 1, pp. 100–118, 1995.
- [17] W.-K. Chen, Ed., *The VLSI Handbook*, 1st ed. CRC Press, 1999.
- [18] B. Taskin and I. S. Kourtev, "Delay insertion method in clock skew scheduling," *IEEE Transactions in Computer-Aided Design of Integrated Circuits and Systems*, vol. 25, no. 4, pp. 651–663, April 2006.
- [19] GLPK (GNU Linear Programming Kit), Free Software Foundation (FSF), <http://www.gnu.org/software/glpk/glpk.html>, 2008.